# SOFTWARE ARCHITECTURE

## Cryptography

BEYOND BOUNDARIES

HOA SEN UNIVERSITY
WORLD CLASS EDUCATION

QS STARS RATING SYSTEM

ACBSP ACCREDITED

AUN-QA

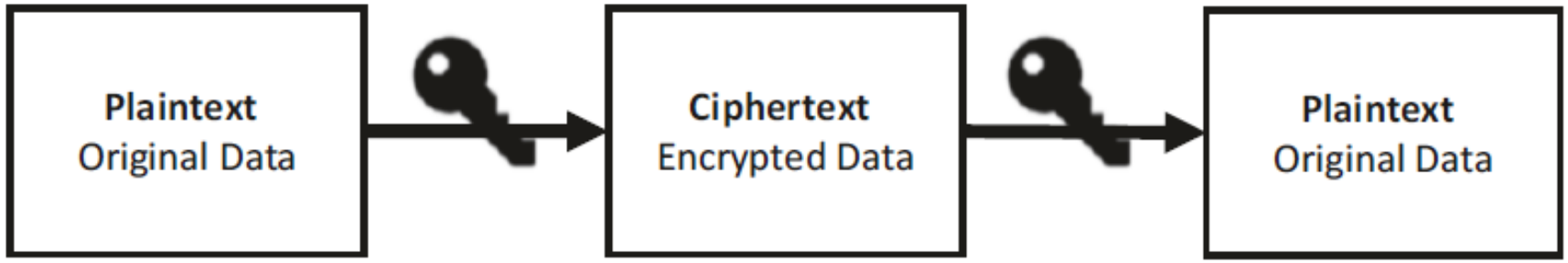
MOET ACCREDITED
KIỂM ĐỊNH CHẤT LƯỢNG GIÁO DỤC QUỐC GIA

# Content

- Symmetric Encryption
- Asymmetric Encryption

# Symmetric Encryption

- Symmetric encryption algorithms are a two-way operation where you use the same key for both encryption and decryption of your message.
- You can reverse the encryption process to recover the original data (if you use the same key), which is why it is referred to as **symmetric**.



# Advantage

- Very Secure:
  - Symmetric encryption is exceptionally secure
  - AES is unbroken, so it is one of the recommended algorithms.
- Fast
  - Encrypting and decrypting symmetric key data is easier to do, giving you excellent read and write performance.
  - In fact, many **solid-state drives (SSD)**, which are very fast, use symmetric key encryption store data, yet they are still a lot faster than unencrypted standard hard drives.

# Disadvantage

- Sharing Keys Is Hard
  - You need to have a way to get the key to the person with who you are sending the encrypted data
- Dangerous If Compromised
  - If someone gets hold of one of your symmetric keys, they can decrypt everything encrypted with that key

# Symmetric Encryption algorithms

- Data Encryption Standard (DES), from 1970s
- Triple DES, 1981
- Advanced Encryption Standard (AES), 2001

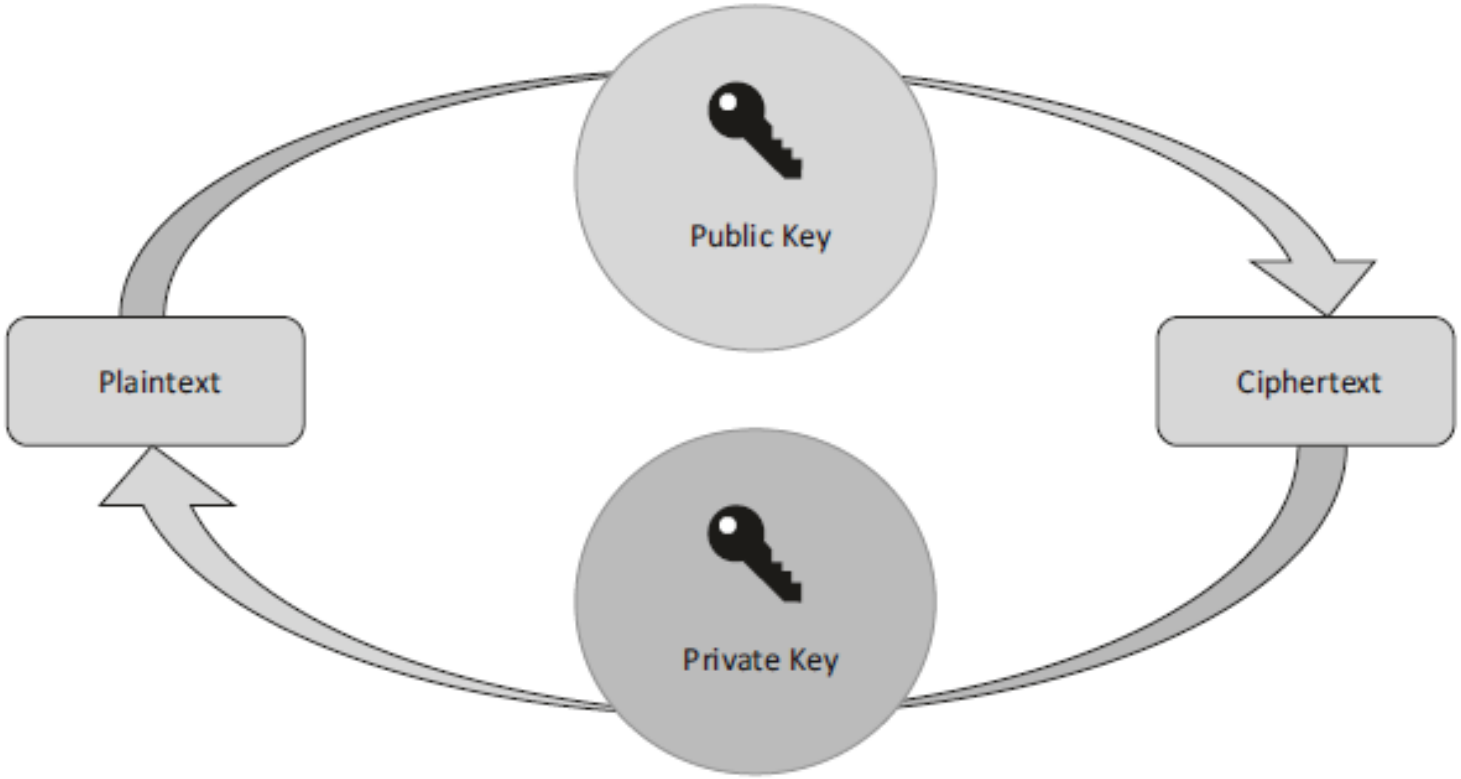
# What Is Asymmetric Encryption?

- Symmetric encryption problem
  - Securely sharing keys.
  - Using the same key to encrypt and decrypt a message
- An excellent solution to this problem is to use asymmetric cryptography, which is also referred to as public key cryptography.

# Public key cryptography

- Have two keys
- A public key, which anyone can know, and a private key, which only the recipient of a message knows.
- These keys are mathematically linked. The message sender uses the public key to encrypt a message and the recipient uses their private key to decrypt the message

# Asymmetric encryption



# Advantage

- Two parties don't need to have pre-shared a secret key to communicate using asymmetric encryption.
- The person encrypting a message only needs to know the recipient's public key, which is available to anyone on request.
- Then only the recipient can decrypt the message with their private key.

# Disadvantage

- The main disadvantage is that the asymmetric algorithm is comparatively complex when compared to symmetric encryption
- A messages take longer to encrypt and decrypt.

# Rivest, Shamir, and Adelman (RSA) algorithm

- RSA typically uses three key sizes: 1024 bit, 2048 bit, and 4096 bit.
- By today's standards, you should use at least a 2048-bit key because 1024-bit keys are now considered weak.
- Ideally, use the largest key size, 4096 bits, but 2048 bits is acceptable.

# RSA encryption

- In this situation A wants to send a message x to B in encrypted form.
  - What‘s B public key
  - What‘s A private key

# RSA encryption

- Public key
  - B chooses two (large) primes p and q. He defines N = pq, and he chooses some number e < (p-1)(q-1) that is coprime to (p-1)(q-1). The easiest way to do this is to make e prime, for example, B can set e = 3.
  - **The pair (N, e) is B's public key**. He makes this information freely available to anyone who wants to send him an encrypted message.
  - We assume that p and q are large, so that x < N. If x ≥ N there is a problem as we will be working modulo N, so x will be confused with another message.

# RSA encryption

- Private key
  - A calculates ($x^e$ mod N). This is what she sends to B.
  - There is a number, d, such that $(x^e)^d$ = (x mod N). **This number d is B's private key**. B keeps d secret.
  - The idea is that it is extremely difficult to calculate d from knowledge of (N, e), but easy to calculate it from p and q. So, B can recover x from $x^e$ in a reasonable amount of time, but nobody else can.
  - Remember that ed - 1 = k(p-1)(q-1), for some $k \in N$

# Attack an RSA-encrypted message

- This system allows A to send B a message x encrypted as $x^e$, which B can decrypt by calculating $(x^e)^d$. If a third-party C wants to read this message, they must do one of two things.
  - C can calculate $y^e$ mod N for all y < N. This is linear in the size of N, but computer scientists care about running times in terms of the lengths of inputs in binary. To represent the numbers up to N in binary we need approximately $\log_2 N$ bits, so, as $N = 2^{\log 2N}$, checking N numbers actually uses exponential operations as a function of the binary length of N.
  - C can factor N into p and q and calculate the inverse of e mod (p-1)(q-1). We don't know for sure, but we believe that there is no efficient algorithm for factoring numbers into their prime factors. At least on classical computers. Peter Shor found an efficient algorithm for integer factorization, but this requires large scale quantum computers, which do not currently exist. Certainly, if such an algorithm exists for `normal' computers then it is a closely guarded secret.

# Cryptography in practice

- RSA is not the only public key encryption method in use. There are others, but the common theme is that there must be a process that is easy to perform, but very hard to reverse.
- RSA, uses the fact that it's easy to multiply two primes, but (probably) very hard to factor a number into two prime factors.
- The security of the system is based on the idea that B can broadcast the information needed to encrypt messages, but only B can efficiently decrypt them.
- For real world applications, people usually use a combination of public and private key encryption. Public key encryption is used to transmit information about a private encryption function, which is
- freshly generated for each interaction, between parties, then further messages are exchanged using that.

# Exercise

- Let p = 11 and q = 13. Choose suitable e and d for use in RSA encryption.

# Solution

- We have (p-1)(q-1) = 10 x 12 = 120.
- The smallest prime that doesn't divide 120 is 7, so set e = 7.
- ed - 1 = k(p-1)(q-1) ), for some k $\in$ N

$\Leftrightarrow$7d - 1 = 120k

$\Leftrightarrow$7d = 120k + 1

$\Leftrightarrow$d = (120k+1)/7

# Solution

- Let k=0, and then k=k+1
  - k=0, d=1/7
  - k=1, d=121/7
  - k=2, d=241/7/7
  - k=3, d=361/7/7
  - k=4, d=480/7
  - k=5, d=601/7
  - k=6, d=103
- So, we have suitable e and d is e = 7, d = 103

# Reading

- Look at CrypTool-Online tools at https://www.cryptool.org/en/cto/ to see more algorithms
- You can try somethings on that website

# Q&A

This Photo by Unknown Author is licensed under CC BY-NC-ND